

ADR-002: Capacity ledger

החלטת ארכיטקטורה (ADR)

נוצר על ידי: אלכס • 2026-09-25

תוכן עניינים

Release (expiry or cancellation) .3

Mechanism .2

The invariant (name it, Rule 17) .1

Scope note .5

Custom-cake approval (the cross-context write, see domain-map.md) .4

The invariant (name it, Rule 17) 1

for every `(day, resource)` pair (resource is : `capacity_never_negative`
, or `work_minutes`), `reserved <= total` , at every moment `oven_minutes`
under any concurrency. amit asserts this directly in tests, not just via
route-level checks: the assertion is "run N concurrent reservation attempts
for the last remaining slot, verify exactly the number that fit succeed and
the rest fail cleanly, verify `reserved` never exceeds `total` at any read
".during the race

Mechanism 2

:One table, `capacity_day_ledger` , one row per calendar day

```

day date primary key
oven_minutes_total int not null
oven_minutes_reserved int not null default 0
work_minutes_total int not null
work_minutes_reserved int not null default 0
is_blackout boolean not null default false
check (oven_minutes_reserved >= 0 and oven_minutes_reserved <=
oven_minutes_total)
check (work_minutes_reserved >= 0 and work_minutes_reserved <=
work_minutes_total)

```

:Reservation is a single statement inside the caller's transaction

```

UPDATE capacity_day_ledger
SET oven_minutes_reserved = oven_minutes_reserved + :oven_cost,
    work_minutes_reserved = work_minutes_reserved + :work_cost
WHERE day = :day
    AND oven_minutes_reserved + :oven_cost <= oven_minutes_total
    AND work_minutes_reserved + :work_cost <= work_minutes_total
    AND is_blackout = false
RETURNING day;

```

If this returns zero rows, the reservation failed (insufficient capacity or blackout day) and the caller rolls back the whole order-creation transaction, never partially. The `CHECK` constraints are the second independent line of defense (belt-and-suspenders): even a caller that bypasses the conditional `WHERE` (a bug, not a race) cannot write a negative or over-total row, the transaction aborts instead. This satisfies the learned rule on Storage/RLS-adjacent DB correctness in spirit 2026-04-20 .never trust a single layer for a money/capacity invariant

No `SELECT ... FOR UPDATE` + separate `UPDATE` round trip is used deliberately: a single conditional `UPDATE...WHERE...RETURNING` is atomic in Postgres without an explicit row lock statement, and is one fewer round trip under the two-customers-racing-for-the-last-slot scenario .that is this project's actual production risk

Release (expiry or cancellation) 3

Symmetric decrement, also conditional, also floor-checked by the same

:constraints CHECK

```
UPDATE capacity_day_ledger
SET oven_minutes_reserved = GREATEST(0, oven_minutes_reserved - :oven_cost),
    work_minutes_reserved = GREATEST(0, work_minutes_reserved - :work_cost)
WHERE day = :day;
```

guards against a double-release (e.g., a retried sweep) `GREATEST(0, ...)`

ever taking `reserved` negative, which the `CHECK constraint` would also

.catch, so this is belt-and-suspenders again rather than the only guard

Custom-cake approval (the cross-context write, see domain-map.md) 4

`UPDATE` runs `INSERT INTO orders (...)` and the reservation `approve()`

above in the SAME transaction. If the reservation fails (capacity vanished ,between when Yuval opened the review screen and when she clicked approve

because a standard order took the last slot in the meantime), the whole

,approve transaction rolls back and the admin UI shows "capacity changed

:re-check before approving" rather than silently overbooking (PRD US-2 AC

."never a silent overbook"

Scope note 5

This is deliberately a single-table ledger, not a per-order-line reservations table with a SUM query, because the SUM-query pattern is exactly the race PRD-01 calls out ("two customers checking out for the last slot on the same day must never both succeed") – a read-then-write of a

CHECK derived total has a race window a running total column with a .constraint does not

מאושר

מאושר